объектно ориентированное программирование

комбинации паттернов

комбинации паттернов

- при комбинации всегда есть основной паттерн и вспомогающи(й/е) паттерн(ы)

комбинации паттернов

- при комбинации всегда есть основной паттерн и вспомогающи(й/е) паттерн(ы)

- основной паттерн
  в его домене лежит решаемая проблема;
  с ним взаимодействует вызывающий код;

комбинации паттернов

- при комбинации всегда есть основной паттерн и вспомогающи(й/е) паттерн(ы)

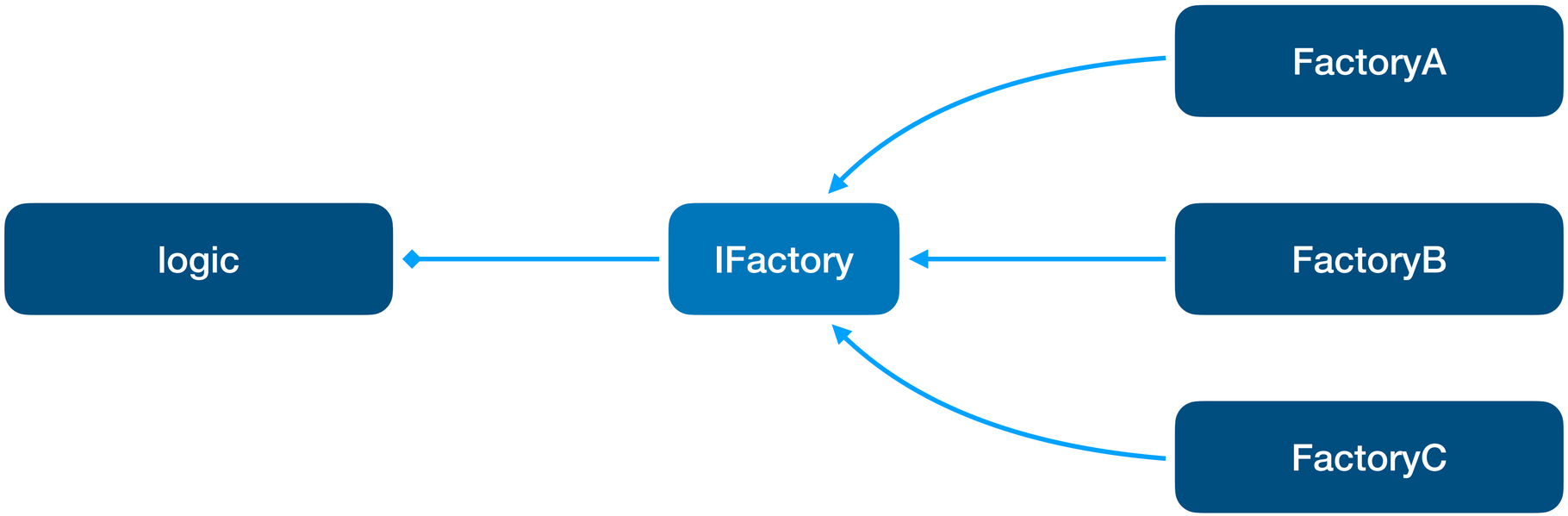
- основной паттерн
  в его домене лежит решаемая проблема;
  с ним взаимодействует вызывающий код;

- вспомогающи(й/е) паттерн(ы)
  является деталью реализации основного паттерна;
  помогает решить проблемы при реализации основного паттерна;

фабрика + …

что смотрели в воркшопе

фабрика + ...



что смотрели в воркшопе

фабрика + …

- разные типы фабрик встраиваются в код
  - то есть решение о вариативности принято извне, до создания объекта с логикой

что ещё может быть

фабрика + …

- разные типы фабрик встраиваются в код
  - то есть решение о вариативности принято извне, до создания объекта с логикой

- что если решение о вариативности нужно принимать в процессе работы объекта с логикой?
  - тогда за это решение ответственна фабрика

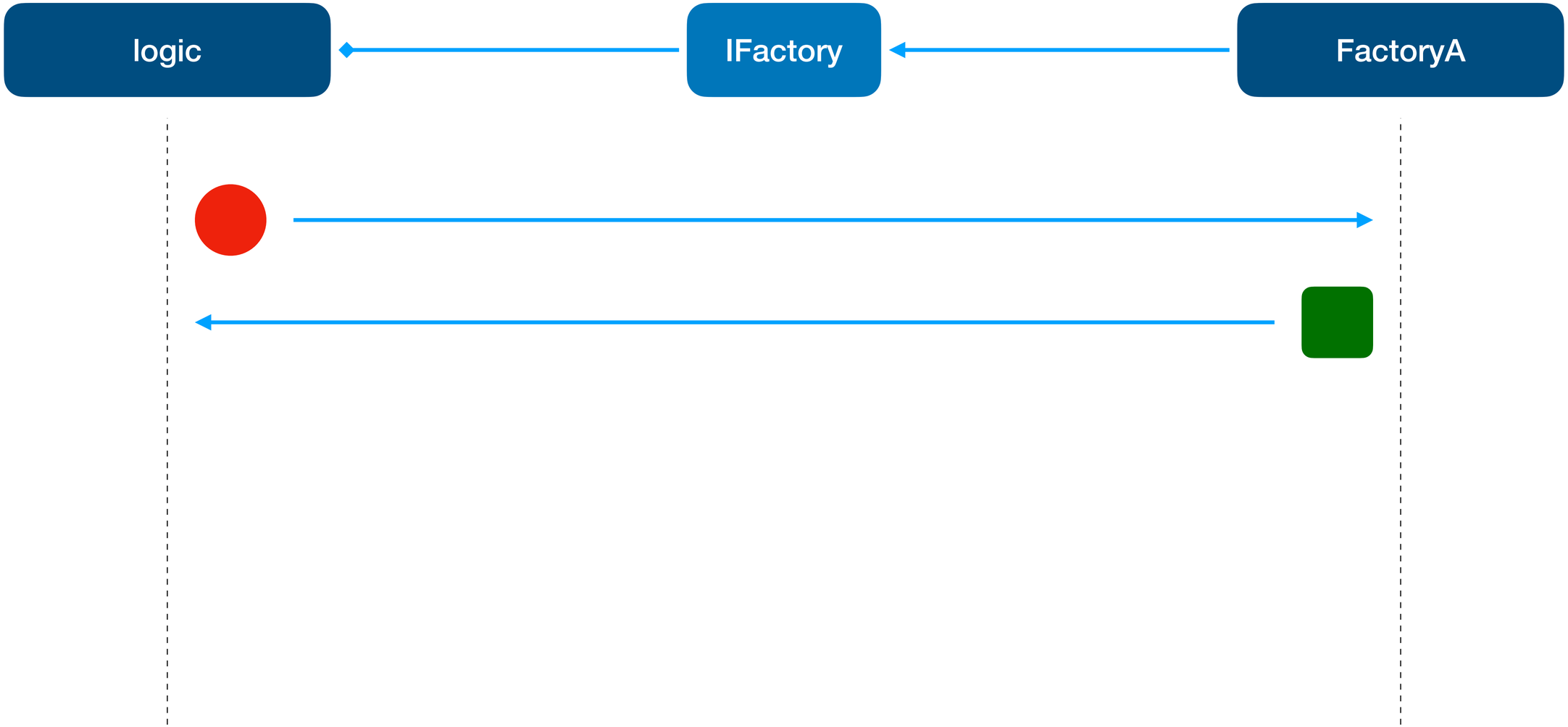
что ещё может быть

фабрика + ...



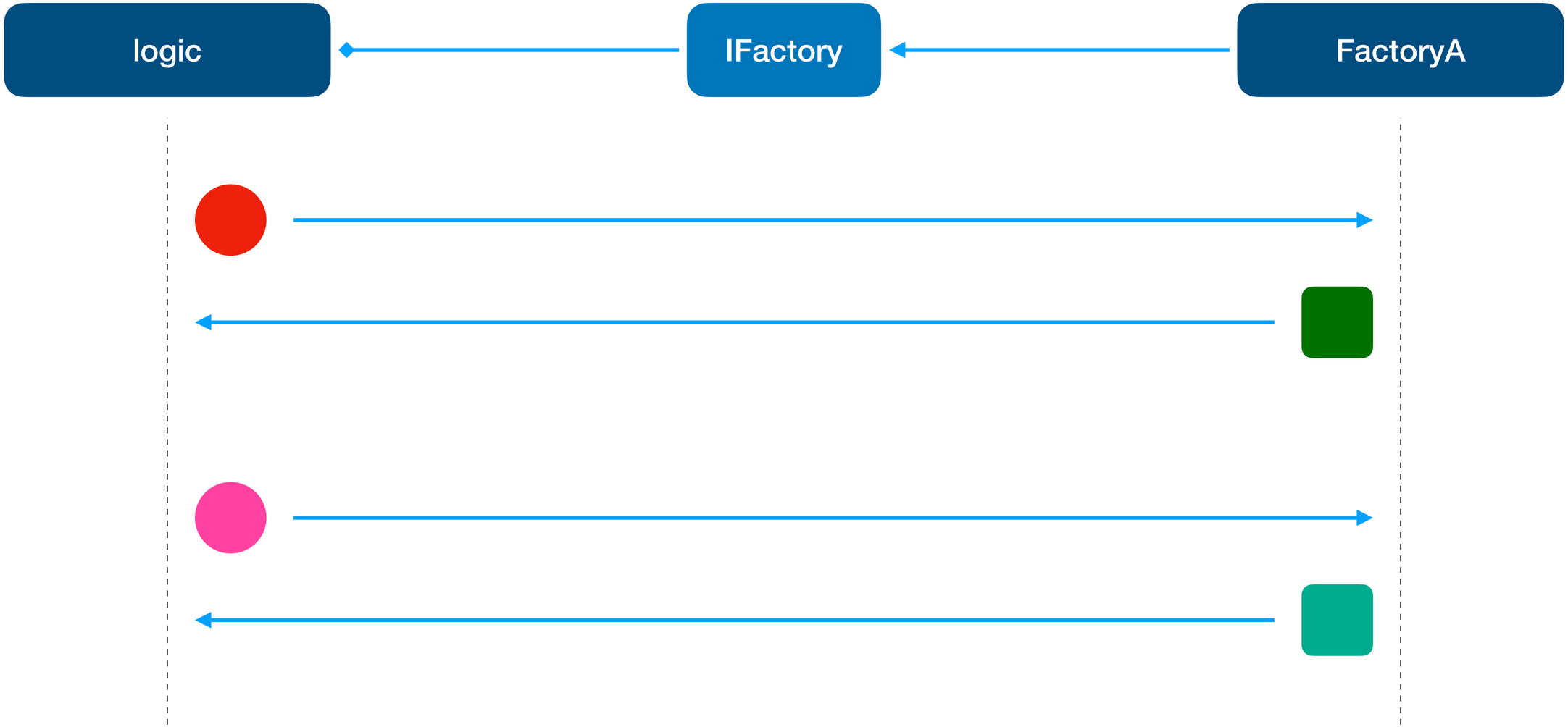
что ещё может быть

фабрика + ...



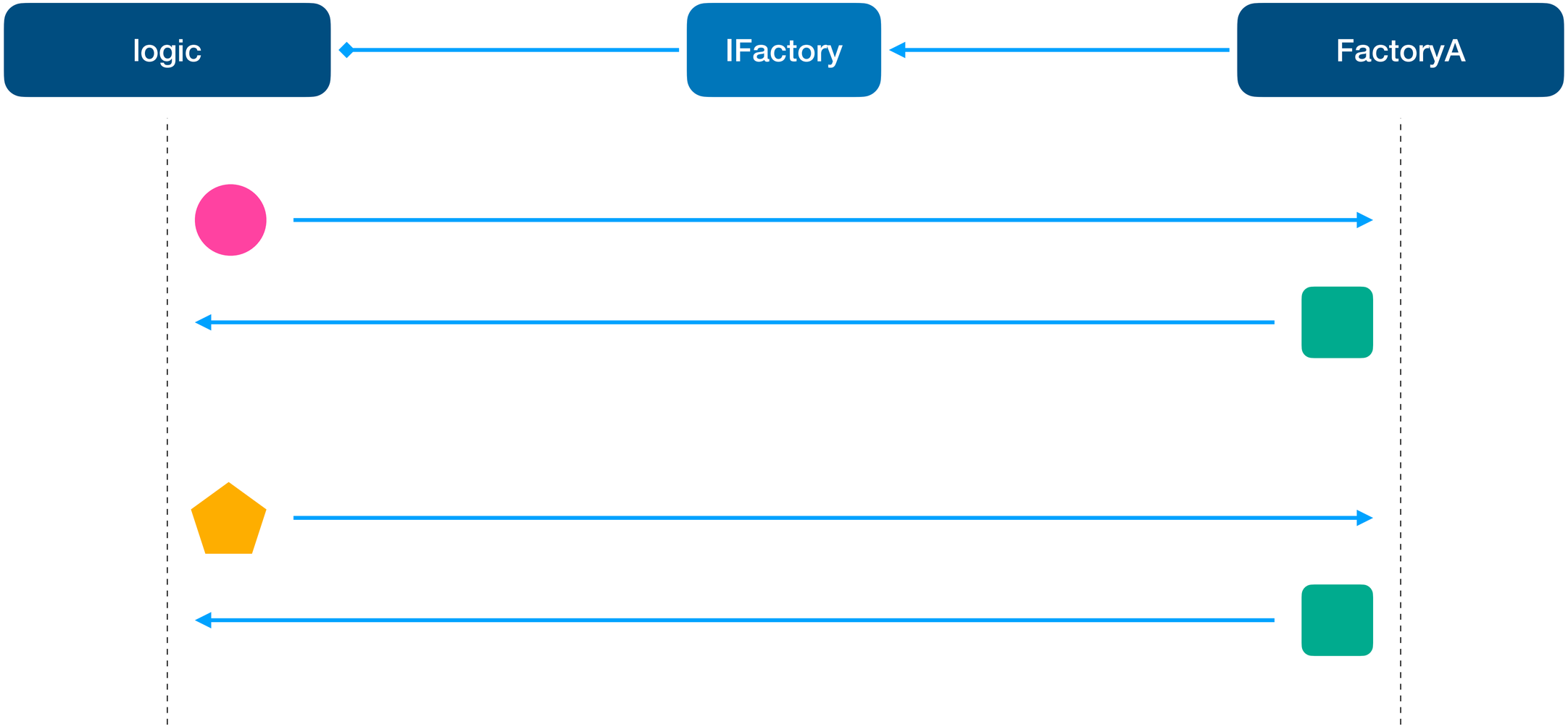
что ещё может быть

фабрика + ...



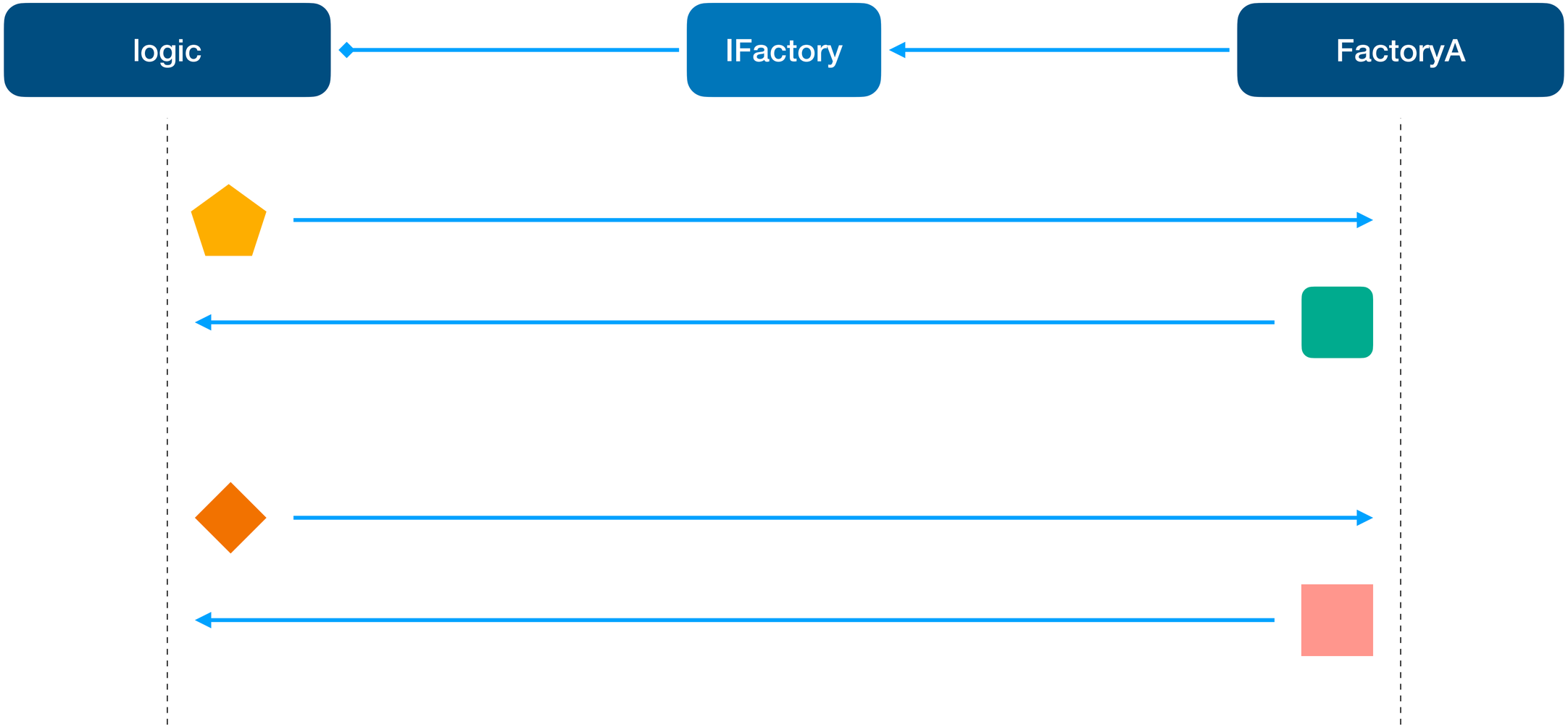
что ещё может быть

фабрика + ...



что ещё может быть

фабрика + ...



проблематика

фабрика + …

- решение о том, как и какой объект создавать может зависеть от множества условий
- фабрика – лучшее место для их реализации
  - но не любая реализация фабрики лучшая

пример

фабрика + ...

```
public sealed record DataRow(...);

public interface IDataParser
{
    IEnumerable<DataRow> ParseRows(Stream source);
}
```

пример

фабрика + ...

```
public sealed record SourceDescriptor(
    string FileType,
    string? CompressionType);

public interface IDataParserFactory
{
    IDataParser Create(SourceDescriptor descriptor);
}
```

пример

фабрика + ...

```
public class DataParserFactory : IDataParserFactory
{
    public IDataParser Create(SourceDescriptor descriptor)
    {
        IDataParser baseParser = descriptor.FileType switch
        {
            MediaTypeNames.Application.Json ⇒ new JsonDataParser(),
            MediaTypeNames.Application.Xml ⇒ new XmlDataParser(),
            MediaTypeNames.Text.Csv ⇒ new CsvDataParser(),

            _ ⇒ throw new NotSupportedException($"'{descriptor.FileType}' file type is not supported"),
        };

        IDataParser decompressionDataParser = descriptor.CompressionType switch
        {
            null ⇒ baseParser,
            MediaTypeNames.Application.Zip ⇒ new ZipDataParserDecorator(baseParser),
            MediaTypeNames.Application.GZip ⇒ new GZipDataParserDecorator(baseParser),

            _ ⇒ throw new NotSupportedException($"'{descriptor.CompressionType}' is not supported"),
        };

        return decompressionDataParser;
    }
}
```

недостатки

фабрика + ...

- нарушение SRP
- нарушение OCP
- отсутствие модульности
  - нет возможности простым образом добавить новый тип парсера
  - нет возможности простым образом добавить новый тип сжатия
  - нет возможности простым образом добавить новый шаг обработки

решение

фабрика + цепочка обязанностей

- комбинация
  - фабрика
  - цепочка обязанностей

решение

фабрика + цепочка обязанностей

- комбинация
  - фабрика
    то, что мы реализуем
  - цепочка обязанностей

решение

фабрика + цепочка обязанностей

- комбинация
  - фабрика
    то, что мы реализуем
  - цепочка обязанностей
    то, как мы это реализуем

решение

фабрика + цепочка обязанностей

```
public abstract class DataParserFactoryStepBase : IDataParserFactory
{
    private DataParserStepBase? _next;

    public abstract IDataParser Create(SourceDescriptor descriptor);

    public DataParserStepBase AddNext(DataParserStepBase next)
    {
        if (_next is null)
        {
            _next = next;
        }
        else
        {
            _next.AddNext(next);
        }

        return this;
    }

    protected IDataParser CreateNext(SourceDescriptor descriptor)
    {
        return _next?.Create(descriptor) ?? throw new InvalidOperationException("Misconfigured factory");
    }
}
```

решение

фабрика + цепочка обязанностей

```
public class JsonDataParserFactoryStep : DataParserFactoryStepBase
{
    public override IDataParser Create(SourceDescriptor descriptor)
    {
        return descriptor.FileType is MediaTypeNames.Application.Json
            ? new JsonDataParser()
            : CreateNext(descriptor);
    }
}

public class XmlDataParserFactoryStep : DataParserFactoryStepBase
{
    public override IDataParser Create(SourceDescriptor descriptor)
    {
        return descriptor.FileType is MediaTypeNames.Application.Xml
            ? new XmlDataParser()
            : CreateNext(descriptor);
    }
}

public class CsvDataParserFactoryStep : DataParserFactoryStepBase
{
    public override IDataParser Create(SourceDescriptor descriptor)
    {
        return descriptor.FileType is MediaTypeNames.Text.Csv
            ? new CsvDataParser()
            : CreateNext(descriptor);
    }
}
```

решение

фабрика + цепочка обязанностей

```
public class ZipDataParserFactoryStep : DataParserFactoryStepBase
{
    public override IDataParser Create(SourceDescriptor descriptor)
    {
        return descriptor.CompressionType is MediaTypeNames.Application.Zip
            ? new ZipDataParserDecorator(CreateNext(descriptor))
            : CreateNext(descriptor);
    }
}

public class GZipDataParserFactoryStep : DataParserFactoryStepBase
{
    public override IDataParser Create(SourceDescriptor descriptor)
    {
        return descriptor.CompressionType is MediaTypeNames.Application.GZip
            ? new GZipDataParserDecorator(CreateNext(descriptor))
            : CreateNext(descriptor);
    }
}
```

решение

фабрика + цепочка обязанностей

```
public class UnsupportedFileTypeDataParserFactoryStep : DataParserFactoryStepBase
{
    public override IDataParser Create(SourceDescriptor descriptor)
        => throw new NotSupportedException($"'{descriptor.FileType}' file type is not supported");
}

public class UnsupportedCompressionTypeDataParserFactoryStep : DataParserFactoryStepBase
{
    public override IDataParser Create(SourceDescriptor descriptor)
    {
        return descriptor.CompressionType is not null
            ? throw new NotSupportedException($"'{descriptor.CompressionType}' is not supported")
            : CreateNext(descriptor);
    }
}
```

решение

фабрика + цепочка обязанностей

```
IDataParserFactory dataParserFactory =
    new ZipDataParserFactoryStep()
        .AddNext(new GZipDataParserFactoryStep())
        .AddNext(new UnsupportedCompressionTypeDataParserFactoryStep())
        .AddNext(new JsonDataParserFactoryStep())
        .AddNext(new XmlDataParserFactoryStep())
        .AddNext(new CsvDataParserFactoryStep())
        .AddNext(new UnsupportedFileTypeDataParserFactoryStep());
```

решение

фабрика + цепочка обязанностей

```
public abstract class DataParserFactoryStepBase : IDataParserFactory
{
    ...

    public static DataParserFactoryStepBase operator |(
        DataParserFactoryStepBase left,
        DataParserFactoryStepBase right)
    {
        return left.AddNext(right);
    }

    ...
}
```

решение

фабрика + цепочка обязанностей

```
IDataParserFactory dataParserFactory =
    new ZipDataParserFactoryStep()
    | new GZipDataParserFactoryStep()
    | new UnsupportedCompressionTypeDataParserFactoryStep()
    | new JsonDataParserFactoryStep()
    | new XmlDataParserFactoryStep()
    | new CsvDataParserFactoryStep()
    | new UnsupportedFileTypeDataParserFactoryStep();
```

визитор + …

проблематика

визитор + ...

- пишем логику, которая выполняется поверх всех элементов дерева
- логика разная, но обходим одинаково

проблематика

визитор + ...

```
public interface IFileSystemComponent
{
    string Name { get; }

    void Accept(IFileSystemVisitor visitor);
}
```

```
public interface IFile : IFileSystemComponent
{
    string Content { get; }
}

public interface IDirectory : IFileSystemComponent
{
    IEnumerable<IFileSystemComponent> Children { get; }
}
```

проблематика

визитор + ...

```
public interface IFileSystemVisitor
{
    void Visit(IFile file);

    void Visit(IDirectory directory);
}
```

проблематика

визитор + ...

```
public class FileCountingVisitor : IFileSystemVisitor
{
    public int FileCount { get; private set; }

    public void Visit(IFile file)
    {
        FileCount++;
    }

    public void Visit(IDirectory directory)
    {
        foreach (IFileSystemComponent child in directory.Children)
        {
            child.Accept(this);
        }
    }
}
```

```
public class FileHashingVisitor(
    IncrementalHash hash)
    : IFileSystemVisitor
{
    public void Visit(IFile file)
    {
        hash.AppendData(Encoding.UTF8.GetBytes(file.Name));
        hash.AppendData(Encoding.UTF8.GetBytes(file.Content));
    }

    public void Visit(IDirectory directory)
    {
        hash.AppendData(Encoding.UTF8.GetBytes(directory.Name));

        foreach (IFileSystemComponent child in directory.Children)
        {
            child.Accept(this);
        }
    }
}
```

проблематика

визитор + ...

```
public class FileCountingVisitor : IFileSystemVisitor
{
    public int FileCount { get; private set; }

    public void Visit(IFile file)
    {
        FileCount++;
    }

    public void Visit(IDirectory directory)
    {
        foreach (IFileSystemComponent child in directory.Children)
        {
            child.Accept(this);
        }
    }
}
```

```
public class FileHashingVisitor(
    IncrementalHash hash)
    : IFileSystemVisitor
{
    public void Visit(IFile file)
    {
        hash.AppendData(Encoding.UTF8.GetBytes(file.Name));
        hash.AppendData(Encoding.UTF8.GetBytes(file.Content));
    }

    public void Visit(IDirectory directory)
    {
        hash.AppendData(Encoding.UTF8.GetBytes(directory.Name));

        foreach (IFileSystemComponent child in directory.Children)
        {
            child.Accept(this);
        }
    }
}
```

проблематика

визитор + ...

```
public class FileCountingVisitor : IFileSystemVisitor
{
    public int FileCount { get; private set; }

    public void Visit(IFile file)
    {
        FileCount++;
    }

    public void Visit(IDirectory directory)
    {
        foreach (IFileSystemComponent child in directory.Children)
        {
            child.Accept(this);
        }
    }
}
```

```
public class FileHashingVisitor(
    IncrementalHash hash)
    : IFileSystemVisitor
{
    public void Visit(IFile file)
    {
        hash.AppendData(Encoding.UTF8.GetBytes(file.Name));
        hash.AppendData(Encoding.UTF8.GetBytes(file.Content));
    }

    public void Visit(IDirectory directory)
    {
        hash.AppendData(Encoding.UTF8.GetBytes(directory.Name));

        foreach (IFileSystemComponent child in directory.Children)
        {
            child.Accept(this);
        }
    }
}
```

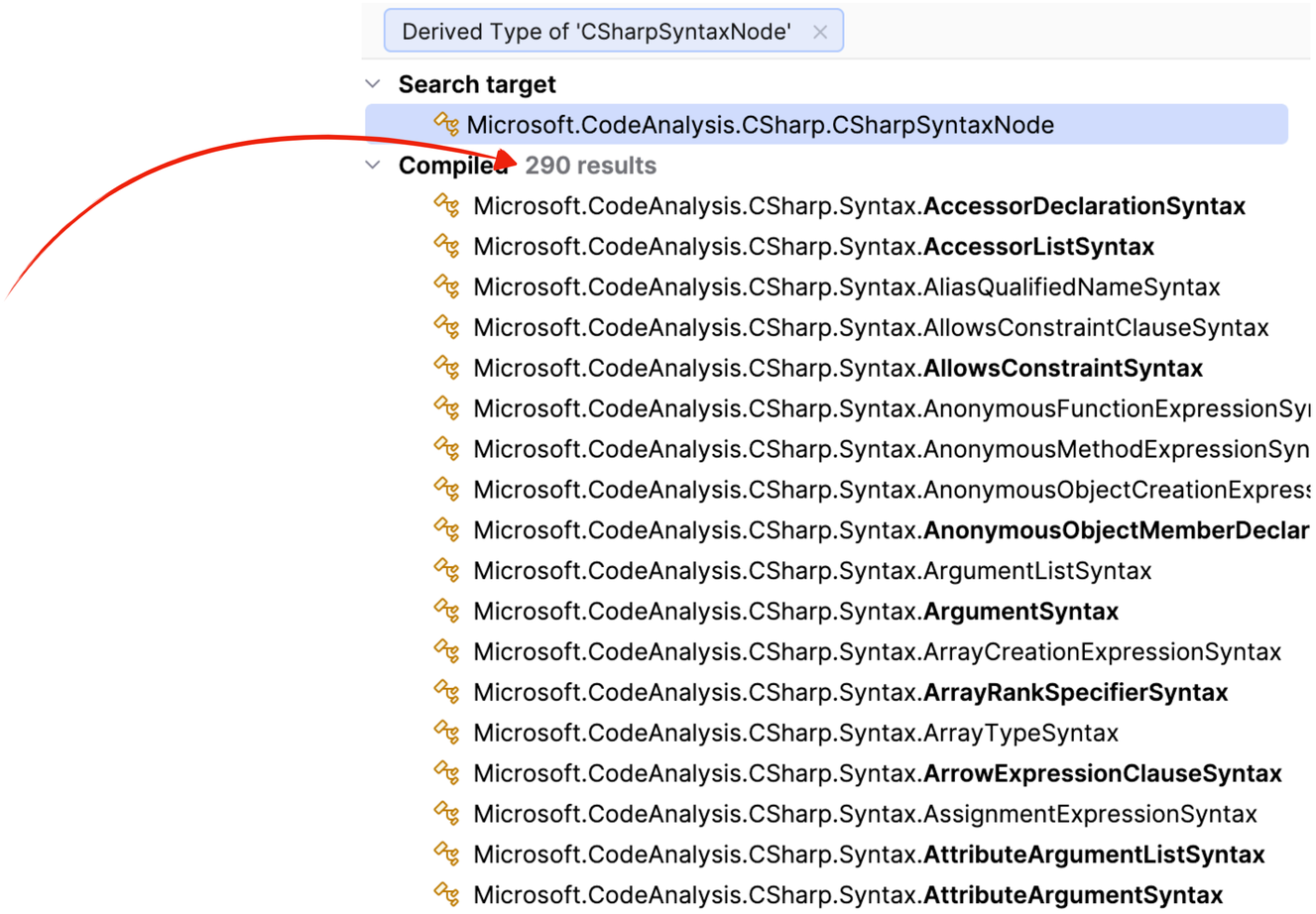
проблематика

визитор + …

- дублируем логику обхода дерева
  - больше пространства для ошибок
  - чем больше разных компонентов – тем больше проблем

проблематика

визитор + ...



решение

визитор + шаблонный метод

- комбинация
  - визитор
  - шаблонный метод

решение

визитор + шаблонный метод

- комбинация
  - визитор
  - шаблонный метод
    базовый класс – логика обхода
    дочерние классы – логика обработки узла
- общее название – Walker (TreeWalker/SyntaxWalker)

решение

визитор + шаблонный метод

```
public abstract class FileSystemWalker : IFileSystemVisitor
{
    public virtual void Visit(IFile file) { }

    public virtual void Visit(IDirectory directory)
    {
        foreach (IFileSystemComponent child in directory.Children)
        {
            child.Accept(this);
        }
    }
}
```

решение

визитор + шаблонный метод

```
public class FileCountingVisitor : IFileSystemVisitor
{
    public int FileCount { get; private set; }

    public void Visit(IFile file)
    {
        FileCount++;
    }

    public void Visit(IDirectory directory)
    {
        foreach (IFileSystemComponent child in directory.Children)
        {
            child.Accept(this);
        }
    }
}
```

```
public class FileCountingVisitor : FileSystemWalker
{
    public int FileCount { get; private set; }

    public override void Visit(IFile file)
    {
        FileCount++;
    }
}
```

решение

визитор + шаблонный метод

```
public class FileHashingVisitor(
    IncrementalHash hash)
    : IFileSystemVisitor
{
    public void Visit(IFile file)
    {
        hash.AppendData(Encoding.UTF8.GetBytes(file.Name));
        hash.AppendData(Encoding.UTF8.GetBytes(file.Content));
    }

    public void Visit(IDirectory directory)
    {
        hash.AppendData(Encoding.UTF8.GetBytes(directory.Name));

        foreach (IFileSystemComponent child in directory.Children)
        {
            child.Accept(this);
        }
    }
}
```

```
public class FileHashingVisitor(
    IncrementalHash hash)
    : FileSystemWalker
{
    public override void Visit(IFile file)
    {
        hash.AppendData(Encoding.UTF8.GetBytes(file.Name));
        hash.AppendData(Encoding.UTF8.GetBytes(file.Content));
    }

    public override void Visit(IDirectory directory)
    {
        hash.AppendData(Encoding.UTF8.GetBytes(directory.Name));
        base.Visit(directory);
    }
}
```

решение

визитор + шаблонный метод

- вариант 1
  позволяем вызывающему коду иметь возможность контроля обхода

решение

визитор + шаблонный метод

- вариант 1
  позволяем вызывающему коду иметь возможность контроля обхода
- вариант 2
  не позволяем

решение

визитор + шаблонный метод

```
public abstract class FileSystemWalker : IFileSystemVisitor
{
    void IFileSystemVisitor.Visit(IFile file)
    {
        Visit(file);
    }

    void IFileSystemVisitor.Visit(IDirectory directory)
    {
        Visit(directory);

        foreach (IFileSystemComponent child in directory.Children)
        {
            child.Accept(this);
        }
    }

    protected virtual void Visit(IFile file) { }

    protected virtual void Visit(IDirectory directory) { }
}
```

решение

визитор + шаблонный метод

```
public class FileHashingVisitor(
    IncrementalHash hash)
    : FileSystemWalker
{
    public override void Visit(IFile file)
    {
        hash.AppendData(Encoding.UTF8.GetBytes(file.Name));
        hash.AppendData(Encoding.UTF8.GetBytes(file.Content));
    }

    public override void Visit(IDirectory directory)
    {
        hash.AppendData(Encoding.UTF8.GetBytes(directory.Name));
        base.Visit(directory);
    }
}
```

```
public class FileHashingVisitor(
    IncrementalHash hash)
    : FileSystemWalker
{
    protected override void Visit(IFile file)
    {
        hash.AppendData(Encoding.UTF8.GetBytes(file.Name));
        hash.AppendData(Encoding.UTF8.GetBytes(file.Content));
    }

    protected override void Visit(IDirectory directory)
    {
        hash.AppendData(Encoding.UTF8.GetBytes(directory.Name));
    }
}
```

решение

визитор + шаблонный метод

- вариант 1
  позволяем вызывающему коду иметь возможность контроля обхода
- вариант 2
  не позволяем
- вариант 3
  стратегия

решение

визитор + стратегия

```
public sealed class FileSystemWalker(
    IFileSystemVisitor visitor)
    : IFileSystemVisitor
{
    void IFileSystemVisitor.Visit(IFile file)
    {
        visitor.Visit(file);
    }

    void IFileSystemVisitor.Visit(IDirectory directory)
    {
        visitor.Visit(directory);

        foreach (IFileSystemComponent child in directory.Children)
        {
            child.Accept(this);
        }
    }
}
```

решение

визитор + стратегия

```
public class FileCountingVisitor : IFileSystemVisitor
{
    public int FileCount { get; private set; }

    public void Visit(IFile file) ⇒ FileCount++;

    public void Visit(IDirectory directory) { }
}
```

```
public class FileHashingVisitor(
    IncrementalHash hash)
    : IFileSystemVisitor
{
    public void Visit(IFile file)
    {
        hash.AppendData(Encoding.UTF8.GetBytes(file.Name));
        hash.AppendData(Encoding.UTF8.GetBytes(file.Content));
    }

    public void Visit(IDirectory directory)
    {
        hash.AppendData(Encoding.UTF8.GetBytes(directory.Name));
    }
}
```

решение

визитор + шаблонный метод/стратегия

- вариант 1
  + не завязываемся на “встраиваемую” логику
  – может быть не очевидно что нужно вызвать базовый класс (обход может не обойти)

- вариант 2
  + гарантируем обход
  – “встраиваемая” логика тоже может обходить (сами дураки)

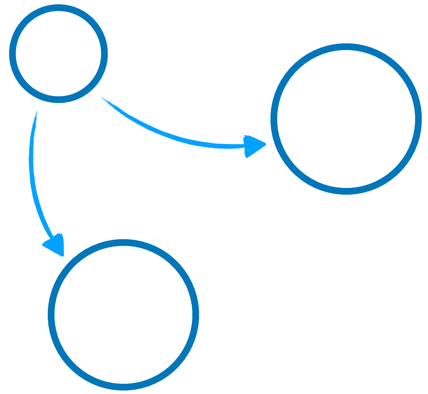
- вариант 3
  + можно использовать логику с разными способами обхода
  – “встраиваемая” логика не самодостаточна

КОМПОНОВЩИК + …

проблематика

компоновщик + …

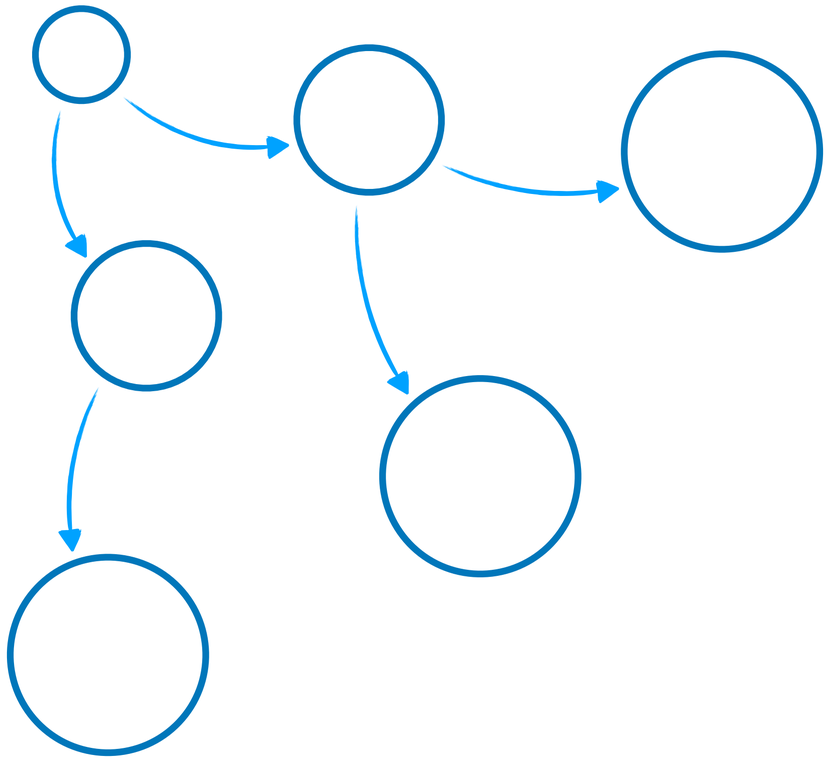
- работаем с деревом



проблематика

компоновщик + …

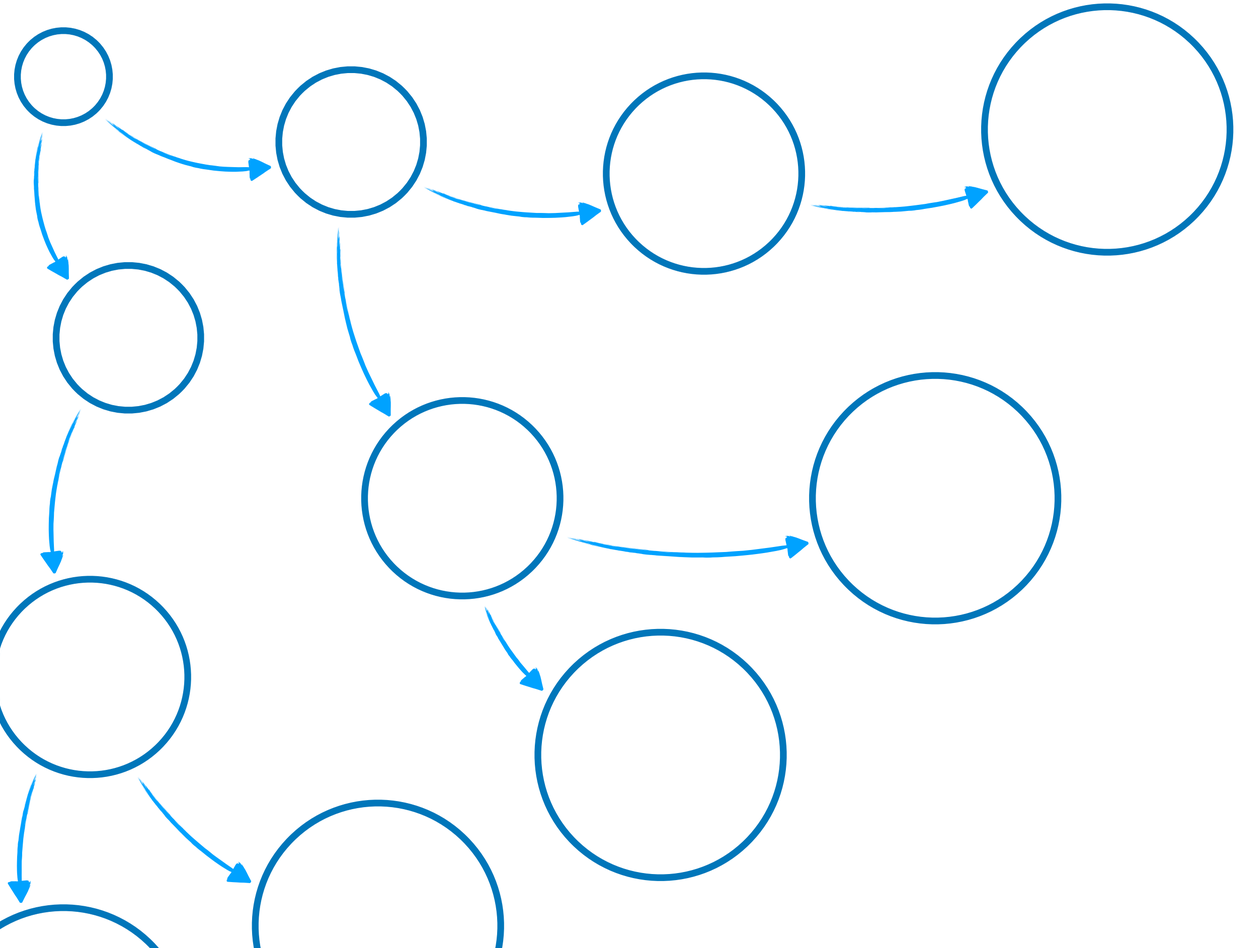
- работаем с большим деревом



проблематика

компоновщик + …

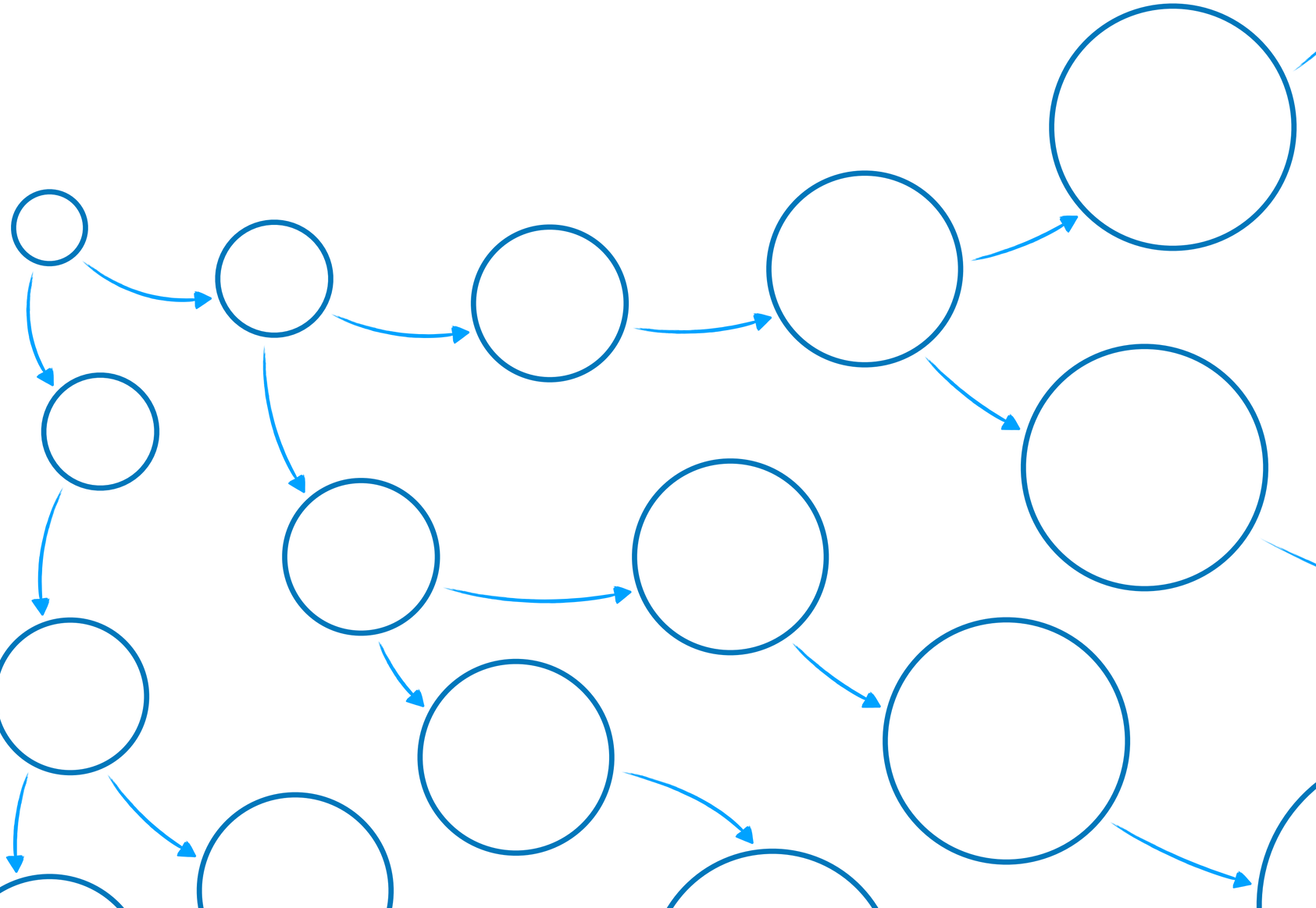
- работаем с очень большим деревом



проблематика

компоновщик + …

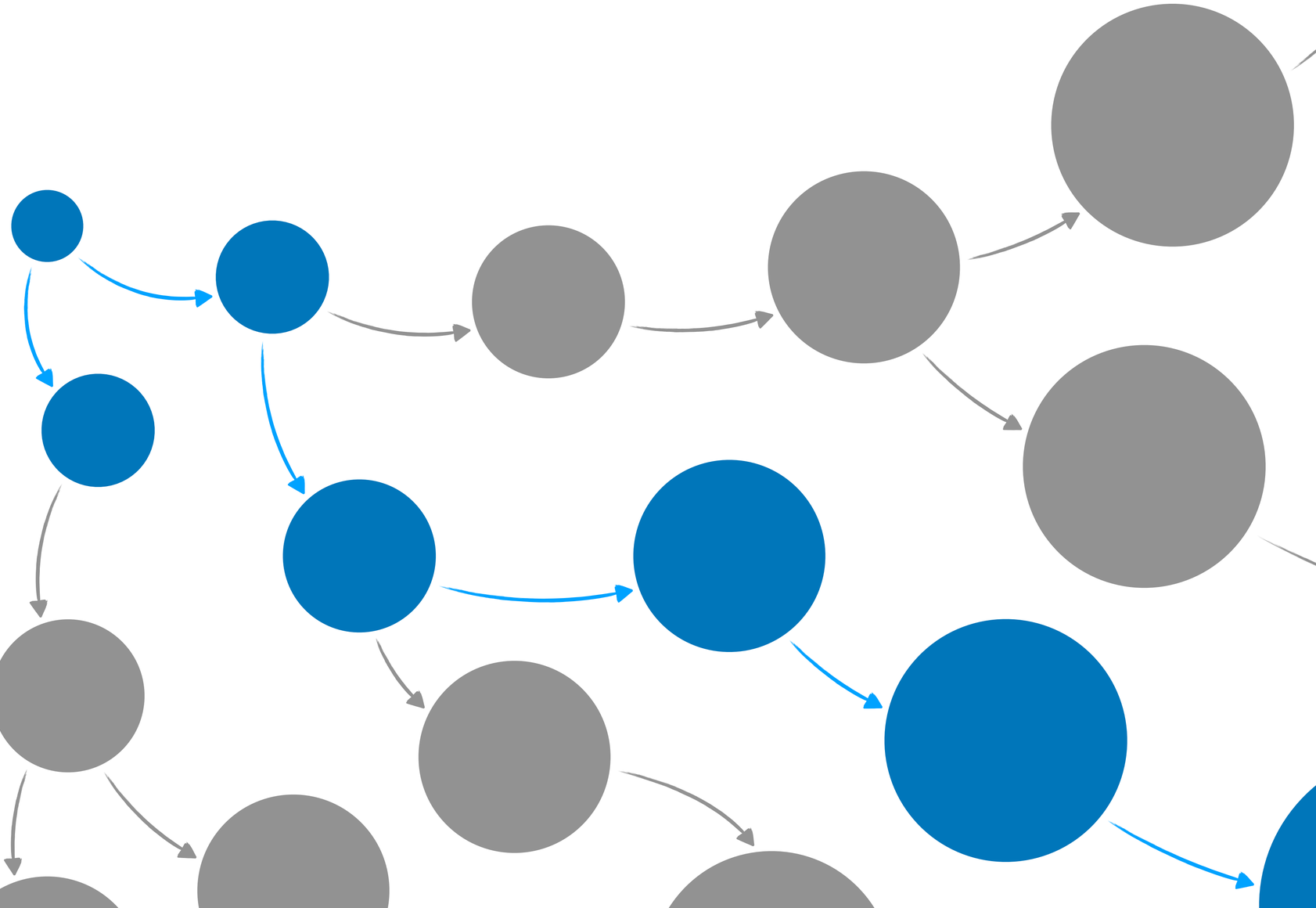
- работаем с очень-очень большим деревом



проблематика

компоновщик + …

- работаем с очень-очень большим деревом
- но при работе нужно сильно меньше данных



проблематика

компоновщик + …

- работаем с очень-очень большим деревом
- но при работе нужно сильно меньше данных

- загружаем всё и сразу – тратим больше ресурсов чем нужно

решение

компоновщик + прокси

- комбинация
  - компоновщик
  - абстрактная фабрика
  - прокси

решение

компоновщик + прокси

- комбинация
  - компоновщик
    нужен для представления деревьев
  - абстрактная фабрика
  - прокси

решение

компоновщик + прокси

- комбинация
  - компоновщик
    нужен для представления деревьев
  - абстрактная фабрика
    нужна для переиспользования логики загрузки элементов дерева
  - прокси

решение

компоновщик + прокси

- комбинация
  - компоновщик
    нужен для представления деревьев
  - абстрактная фабрика
    нужна для переиспользования логики загрузки элементов дерева
  - прокси
    нужен для ленивой загрузки под-деревьев

пример

компоновщик + прокси

```
public interface IEmployeeHierarchyComponent
{
    long Id { get; }

    string Name { get; }

    void Accept(IEmployeeHierarchyVisitor visitor);
}
```

пример

компоновщик + прокси

```
public interface IEmployee : IEmployeeHierarchyComponent
{
    long EmployeeId { get; }
}
```

```
public interface IEmployeeUnit : IEmployeeHierarchyComponent
{
    EmployeeUnitKind Kind { get; }

    IEnumerable<IEmployeeHierarchyComponent> Children { get; }
}
```

пример

компоновщик + прокси

```
public interface IEmployeeHierarchyVisitor
{
    void Visit(IEmployee employee);

    void Visit(IEmployeeUnit unit);
}
```

пример

компоновщик + прокси

- мы хотим получить информацию о цепочке иерархии сотрудника
  - все элементы иерархии между ним и корнем

пример

компоновщик + прокси

```
public class EmployeeHierarchyChainVisitor(
    IReadOnlyList<long> hierarchyChainIds)
    : IEmployeeHierarchyVisitor
{
    private readonly List<IEmployeeHierarchyComponent> _chain = [];
    private int _index;

    public bool IsSuccess => _chain[^1].Id == hierarchyChainIds[^1];
    public IEnumerable<IEmployeeHierarchyComponent> Chain => _chain;

    public void Visit(IEmployee employee)
    {
        if (hierarchyChainIds[_index] == employee.Id)
        {
            _chain.Add(employee);
        }
    }

    public void Visit(IEmployeeUnit unit)
    {
        IEmployeeHierarchyComponent? matchingComponent = unit.Children
            .SingleOrDefault(component => component.Id == hierarchyChainIds[_index]);

        if (matchingComponent is not null)
        {
            _index++;
            matchingComponent.Accept(this);
        }
    }
}
```

пример

компоновщик + прокси

- мы хотим получить информацию о цепочке иерархии сотрудника
  - все элементы иерархии между ним и корнем

- результат – одна длинная цепочка
  - но грузим всё дерево

наивная реализация

компоновщик + прокси

- реализовать логику ленивости в HierarchyUnit
  - нарушаем SRP
  - если нужно будет в новом узле – копипаста

решение

компоновщик + прокси

```
public class LazyEmployeeComponentNode(
    long id,
    IEmployeeComponentFactory factory)
    : IEmployeeHierarchyComponent
{
    private readonly Lazy<IEmployeeHierarchyComponent> _component =
        new(() => factory.GetById(id));

    public long Id { get; } = id;

    public string Name => _component.Value.Name;

    public void Accept(IEmployeeHierarchyVisitor visitor)
    {
        _component.Value.Accept(visitor);
    }
}
```

решение

компоновщик + прокси

```
public class EmployeeLazyComponentFactory(
    IEmployeeHierarchyRepository repository)
    : IEmployeeComponentFactory
{
    public IEmployeeHierarchyComponent GetById(long hierarchyId)
    {
        EmployeeComponentDataModel model = repository.GetById(hierarchyId);

        if (model.Type is EmployeeComponentType.Employee)
        {
            return new Employee(
                model.Id,
                model.Name,
                model.EmployeeId!.Value);
        }

        return new EmployeeUnit(
            model.Id,
            model.Name,
            model.UnitKind!.Value,
            model.ChildIds.Select(id ⇒ new LazyEmployeeComponentNode(id, this)));
    }
}
```

решение

компоновщик + прокси

- мы не будем загружать лишние узлы
- логика ленивой инициализации переиспользуемо

решение

компоновщик + декоратор

```
public class LoggingEmployeeNode(
    IEmployeeHierarchyComponent component,
    ILogger logger)
    : IEmployeeHierarchyComponent
{
    public long Id ⇒ component.Id;

    public string Name ⇒ component.Name;

    public void Accept(IEmployeeHierarchyVisitor visitor)
    {
        logger.Log($"Visiting hierarchy component '{Id}'");
        component.Accept(visitor);
    }
}
```

почему это работает

компоновщик + прокси/декоратор

- прокси/декоратор встраиваются в компоновщик бесшовно
  - нам не нужно добавлять новый метод визитора
  - нам не нужно знать в визиторах про все прокси/декораторы
- абстрактная фабрика разгружает логику прокси
  - сам класс прокси – простой
  - логику фабрики можно переиспользовать (например для создания изначального узла)

команда + …

проблематика

команда + ...

```
public interface IDocumentCommand
{
    DocumentCommandResult Execute(DocumentCommandExecutionContext context);
}
```

проблематика

команда + ...

```
public class ReadDocumentCommand : IDocumentCommand
{
    public DocumentCommandResult Execute(
        DocumentCommandExecutionContext context)
    {
        if (context.User.IsAdmin is false
            && context.User.UserId =/= context.Document.OwnerId)
        {
            return new DocumentCommandResult.Unauthorized();
        }

        ...
    }
}
```

```
public class DuplicateDocumentCommand : IDocumentCommand
{
    public DocumentCommandResult Execute(
        DocumentCommandExecutionContext context)
    {
        if (context.User.IsAdmin is false
            && context.User.UserId =/= context.Document.OwnerId)
        {
            return new DocumentCommandResult.Unauthorized();
        }

        ...
    }
}
```

проблематика

команда + ...

```
public class EditDocumentCommand : IDocumentCommand
{
    public DocumentCommandResult Execute(
        DocumentCommandExecutionContext context)
    {
        if (context.User.IsAdmin is false
            && context.User.UserId != context.Document.OwnerId)
        {
            return new DocumentCommandResult.Unauthorized();
        }

        if (context.Document.IsFrozen)
        {
            return new DocumentCommandResult.Failure("Document is frozen");
        }

        ...
    }
}
```

```
public class DeleteDocumentCommand : IDocumentCommand
{
    public DocumentCommandResult Execute(
        DocumentCommandExecutionContext context)
    {
        if (context.User.IsAdmin is false
            && context.User.UserId != context.Document.OwnerId)
        {
            return new DocumentCommandResult.Unauthorized();
        }

        if (context.Document.IsFrozen)
        {
            return new DocumentCommandResult.Failure("Document is frozen");
        }

        ...
    }
}
```

проблематика

команда + …

- дубликация логики предусловий
  - правила валидации в какой-то комбинации будут дублироваться на всех командах
- дубликация логики команд (в лучшем случае)
  - если от контекста создания команды зависит логика предусловий
- усложнение логики команды (в худшем случае)
  - если вариативность логики предусловий засунуть в одну команду

решение

команда + прокси

- комбинация
  - команда
  - прокси

решение

команда + прокси

- комбинация
  - команда
    как основной паттерн, основной интерфейс
  - прокси

решение

команда + прокси

- комбинация
  - команда
    как основной паттерн, основной интерфейс
  - прокси
    то, как мы выносим общую логику

решение

команда + прокси

```
public class DocumentCommandAccessPrecondition(
    IDocumentCommand command)
    : IDocumentCommand
{
    public DocumentCommandResult Execute(
        DocumentCommandExecutionContext context)
    {
        if (context.User.IsAdmin is false
            && context.User.UserId =/= context.Document.OwnerId)
        {
            return new DocumentCommandResult.Unauthorized();
        }

        return command.Execute(context);
    }
}
```

```
public class DocumentCommandNotFrozenPrecondition(
    IDocumentCommand command)
    : IDocumentCommand
{
    public DocumentCommandResult Execute(
        DocumentCommandExecutionContext context)
    {
        if (context.Document.IsFrozen)
        {
            return new DocumentCommandResult.Failure("Document is frozen");
        }

        return command.Execute(context);
    }
}
```

решение

команда + прокси

```
IDocumentCommand readCommand = new ReadDocumentCommand();
readCommand = new DocumentCommandAccessPrecondition(readCommand);

IDocumentCommand duplicateCommand = new DuplicateDocumentCommand();
duplicateCommand = new DocumentCommandAccessPrecondition(duplicateCommand);


IDocumentCommand editCommand = new EditDocumentCommand();
editCommand = new DocumentCommandNotFrozenPrecondition(editCommand);
editCommand = new DocumentCommandAccessPrecondition(editCommand);

IDocumentCommand deleteCommand = new DeleteDocumentCommand();
deleteCommand = new DocumentCommandNotFrozenPrecondition(deleteCommand);
deleteCommand = new DocumentCommandAccessPrecondition(deleteCommand);
```

итератор + ...

пример

итератор + декоратор

- коллекции и LINQ

```
public interface IEnumerable<T>
{
    IEnumerator<T> GetEnumerator();
}
```

=

итератор + … ?

```
public interface IEnumerable<T>
{
    IEnumerator<T> GetEnumerator();
}
```

=

итератор + фабрика

```
public class SelectIterator<TFrom, TTo>(
    IEnumerator<TFrom> enumerator,
    Func<TFrom, TTo> selector)
    : IEnumerator<TTo>
{
    private TTo? _current;

    public TTo Current ⇒
        _current
        ?? throw new InvalidOperationException();

    public bool MoveNext()
    {
        if (enumerator.MoveNext() is false)
            return false;

        _current = selector(enumerator.Current);
        return true;
    }
}
```

=

итератор + … + …?

```
public class SelectIterator<TFrom, TTo>(
    IEnumerator<TFrom> enumerator,
    Func<TFrom, TTo> selector)
    : IEnumerator<TTo>
{
    private TTo? _current;

    public TTo Current ⇒
        _current
        ?? throw new InvalidOperationException();

    public bool MoveNext()
    {
        if (enumerator.MoveNext() is false)
            return false;

        _current = selector(enumerator.Current);
        return true;
    }
}
```

= итератор + декоратор + стратегия

```
IEnumerable<int> first = [1, 2, 3];
IEnumerable<int> second = [2, 3, 4];

IEnumerable<int> result = first.Intersect(second);

foreach (int value in result)
{
    Console.Write($"{value} "); // 2 3
}
```



=

итератор + … ?

```
IEnumerable<int> first = [1, 2, 3];
IEnumerable<int> second = [2, 3, 4];

IEnumerable<int> result = first.Intersect(second);

foreach (int value in result)
{
    Console.Write($"{value} "); // 2 3
}
```

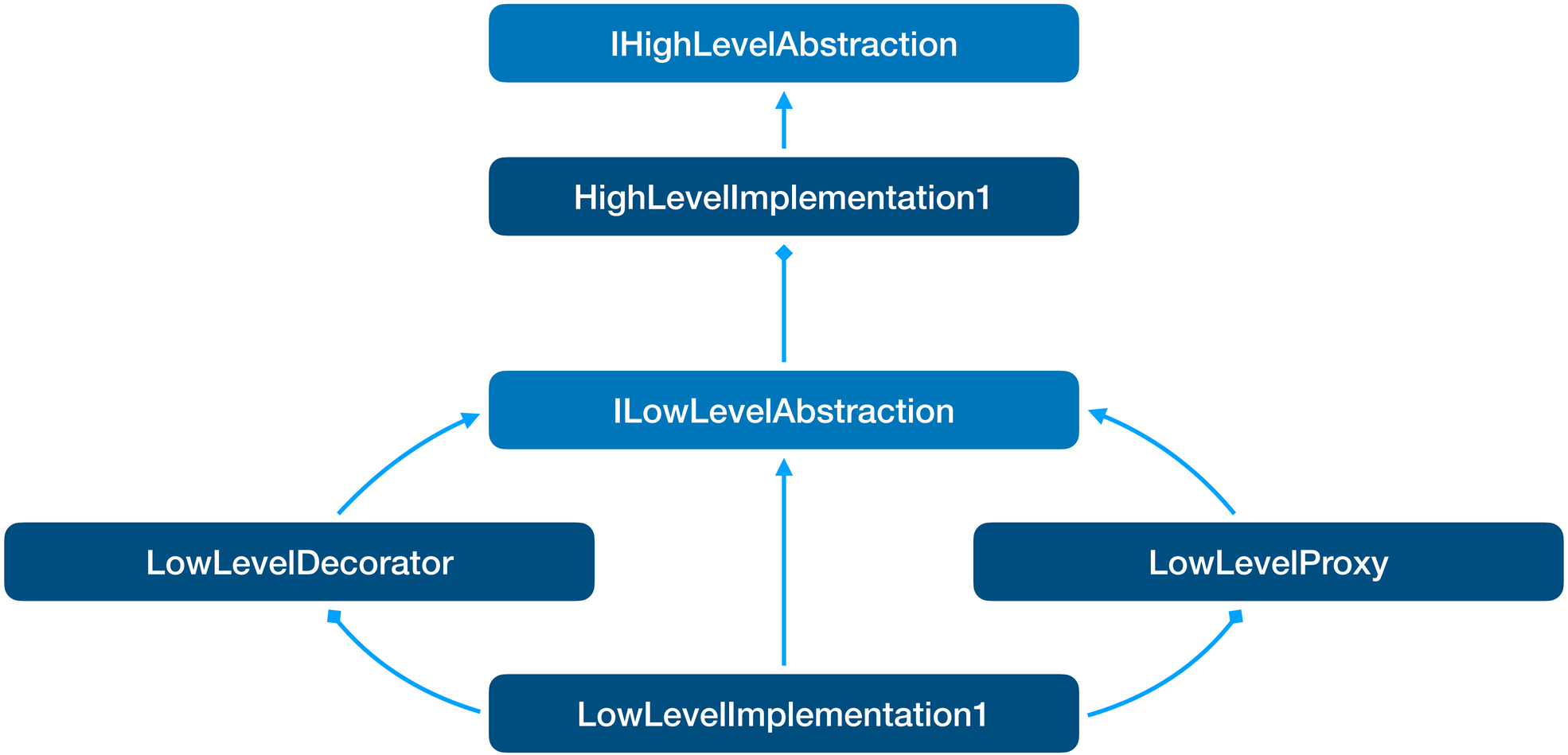


= итератор + компоновщик

другое

мост + декоратор/прокси

другое



билдер + стратегия

другое

```
public interface IComputerConfigurationValidator
{
    IEnumerable<string> Validate(ComputerConfigurationValidationArgs args);
}
```

```
public class ComputerBuilder(
    IEnumerable<IComputerConfigurationValidator> validators)
{
    public ComputerBuildResult Build()
    {
        var args = new ComputerConfigurationValidationArgs(...);

        string[] validationErrors = validators
            .SelectMany(validator => validator.Validate(args))
            .ToArray();

        if (validationErrors is not [])
        {
            return new ComputerBuildResult.Failure(string.Join(", ", validationErrors));
        }

        ...
    }
}
```

прокси + стратегия

другое

```
public interface IAccessValidator
{
    bool ValidateUserAccess();
}
```

```
public class FileAccessorProxy(
    IFileAccessor accessor,
    IAccessValidator validator)
    : IFileAccessor
{
    public string GetContent()
    {
        if (validator.ValidateUserAccess() is false)
            throw new UnauthorizedAccessException();

        return accessor.GetContent();
    }
}
```

прокси + фабрика

другое

```
public interface IFileAccessorFactory
{
    IFileAccessor CreateAccessor(string path);
}
```

```
public class FileAccessorProxy(
    string path,
    IFileAccessorFactory factory)
    : IFileAccessor
{
    public IFileAccessor Accessor => field ??= factory.CreateAccessor(path);

    public string GetContent()
    {
        return Accessor.GetContent();
    }
}
```

наблюдатель + декоратор + стратегия

другое

```
public class SelectObservable<TFrom, TTo>(
    IObservable<TFrom> observable,
    Func<TFrom, TTo> selector)
    : IObservable<TTo>
{
    public IDisposable Subscribe(IObserver<TTo> observer)
    {
        var wrapper = new SelectObserver<TFrom, TTo>(observer, selector);
        return observable.Subscribe(wrapper);
    }
}
```

```
public class SelectObserver<TFrom, TTo>(
    IObserver<TTo> observer,
    Func<TFrom, TTo> selector)
    : IObserver<TFrom>
{
    public void OnCompleted() => observer.OnCompleted();

    public void OnError(Exception error) => observer.OnError(error);

    public void OnNext(TFrom value) => observer.OnNext(selector(value));
}
```